

Proyecto II introducción a la
programación

II semestre 2025

Estudiante: Job Jimenez Sandi – 2023044775 Profesor:

Jeff Schmitt Peralta

documentación del código

Objetivos:

El objetivo principal del proyecto es hacer un videojuego de laberinto en Python que sirva como práctica de varios temas del curso: programación estructurada, programación orientada a objetos, uso de matrices, manejo de archivos y librerías externas.

A nivel de juego, se quiere que el programa tenga dos modos claros (Escapa y Cazador), que haya diferentes tipos de casillas (camino, muro, túnel y liana), enemigos con comportamientos diferentes según el modo, un sistema de energía y correr, trampas limitadas, vidas y un sistema de puntajes con Top 5 por modo. Todo esto debe verse en una interfaz gráfica hecha con Pygame, sin depender de la consola más que para ejecutar main.

Como objetivos de aprendizaje, se busca:

- Usar bien los conceptos básicos de orientación a objetos en Python (clases, objetos, herencia, etc.) [1]–[3].
- Organizar el código en paquetes y módulos para que sea más fácil de entender y mantener [1], [2].
- Aprender a usar Pygame para hacer un bucle de juego con eventos de teclado, dibujo en pantalla y música [4].
- Guardar información en archivos usando JSON como formato de intercambio de datos [5], [6].
- Practicar el uso de Git y GitHub para controlar versiones del proyecto y poder volver atrás si algo sale mal [7].

Arquitectura y diseño orientado a objetos:

La idea general del diseño es que cada cosa importante del enunciado sea una clase o un módulo. El mapa está representado por la clase Mapa, que trabaja con una matriz de objetos Casilla y tiene métodos como obtener_casilla (mapa.py, línea 46), es_transitable_por_jugador(52) y es_transitable_por_enemigo(58). Así, el resto del código no se preocupa por cómo se guarda internamente la matriz, solo le pregunta al mapa si una casilla se puede usar o no. Este estilo de diseño sigue lo que explica la documentación de Python sobre clases y encapsulamiento [1], [2].

Las casillas del mapa se organizan en una pequeña jerarquía: hay una clase base Casilla y subclases para Camino, Muro, Tunel y Liana. Cada una define qué se puede hacer en esa casilla. Esto está inspirado en el uso de herencia y polimorfismo que se ve en los ejemplos de programación orientada a objetos en Python [1]–[3]. En algunas propiedades, como el “código” de la casilla, se usan atributos de solo lectura para evitar cambios accidentales.

El Jugador y el Enemigo comparten la idea de tener una posición (x, y) y un método para moverse, pero luego cada uno tiene su propia lógica. El jugador tiene energía, puede correr, tiene vidas, etc. El enemigo, en cambio, tiene un método actualizar que decide hacia dónde se mueve según el modo de juego: en Escapa persigue al jugador y en Cazador intenta alejarse y

llegar a la salida. Esto sigue la idea de tener clases con responsabilidades claras, como se recomienda en material sobre diseño orientado a objetos [3].

El módulo terrenos se encarga de los túneles y las lianas. Como las casillas no permiten cambiar directamente su tipo (el atributo es de solo lectura), se decidió guardar las posiciones de túneles y lianas en conjuntos (set) y aplicar los efectos después del movimiento. Esta solución aprovecha que los conjuntos en Python permiten verificar rápido si una coordenada está marcada o no [2], y evita romper el diseño original de las clases de casillas.

Interfaz gráfica y flujo del juego con Pygame:

La parte gráfica del juego está hecha con Pygame. Se sigue el esquema clásico que muestran los tutoriales: primero se inicializa Pygame, se crea la ventana, y luego se entra en un bucle donde se leen eventos, se actualiza el estado del juego y se dibuja todo en pantalla antes de hacer `display.flip()` [4].

En la función `ejecutar_juego` (`pygame_ui.py`, línea 121) se ve claramente este patrón:

1. Se leen los eventos con `pygame.event.get()`.
2. Dependiendo de la tecla, el jugador se mueve, cambia entre correr o caminar, coloca trampas, regenera el laberinto, etc.
3. Se actualizan enemigos, trampas, vidas y puntajes.
4. Se revisa si hay victoria o derrota.
5. Se dibuja el mapa, el jugador, los enemigos, las trampas y el HUD.

El manejo de teclado usa la cola de eventos de Pygame (`pygame.event.get()`), que es la forma recomendada para no bloquear la ventana mientras se espera input [4]. Con esto se soportan las teclas WASD o flechas para moverse, R para correr, T para trampas (en el modo Escapa), R en la pantalla previa para regenerar el laberinto y ESC para volver al menú o salir.

El HUD se dibuja usando fuentes (Font) y superficies de texto, tal y como se muestra en los ejemplos de texto de Pygame [4]. En la parte superior se muestra el nombre del jugador, la energía, si está corriendo o no, las vidas, el número de enemigos y trampas activas, y en el modo Cazador el puntaje actual. Para el final de la partida se usan pantallas de “Victoria” y “Game Over” donde se centra el texto, se muestran algunos datos (tiempo, enemigos atrapados, etc.) y se espera a que el jugador presione Enter o Escape.

La música de fondo se controla con el módulo mixer de Pygame. Se carga un archivo de audio, se reproduce en bucle y se configura el volumen [4]. El volumen se puede cambiar desde el menú y se mantiene cuando se entra al juego, de forma que la canción no se reinicia cada vez que empieza una partida nueva.

Mecánicas de juego, terrenos especiales y modos:

El juego usa una grilla (matriz) como base: cada casilla del mapa es una celda en esa matriz, y el jugador y los enemigos ocupan una casilla a la vez. Luego, al dibujar, esas coordenadas se multiplican por el tamaño de celda para conseguir las posiciones en píxeles. Este enfoque es

muy común en ejemplos de juegos 2D con Pygame [4] y hace más sencillo trabajar con colisiones y recorridos.

En el modo Escapa, los enemigos representan cazadores. Tratan de acercarse al jugador con reglas simples de movimiento, y si logran llegar a la misma casilla, el jugador pierde una vida. El jugador puede correr (aumenta su velocidad pero baja la energía) y puede poner trampas hasta un máximo permitido y con un tiempo de recarga. Cuando un enemigo pisa una trampa, muere por un rato y después reaparece en otra parte del laberinto. El jugador gana si llega a la salida y pierde si se queda sin vidas. Estas reglas se implementan en métodos de Jugador, Enemigo y Trampa, siguiendo ideas básicas de movimiento en grilla y detección de colisión por igualdad de coordenadas [3], [4].

En el modo Cazador, todo se da vuelta: ahora el jugador es quien persigue a los enemigos, y ellos tratan de llegar a la salida. Cada vez que el jugador pisa la casilla de un enemigo, lo atrapa y se suma al contador. Si un enemigo logra llegar a la salida, se suma al contador de escapados. El juego termina cuando el jugador ha atrapado a todos o cuando ya no quedan enemigos vivos pero algunos escaparon; en ese caso se considera que no cumplió el objetivo. La ventaja del diseño actual es que para este modo no fue necesario crear nuevas clases, solo cambiar la lógica de actualización de los enemigos y las condiciones de fin de partida [1]–[3].

Los túneles y las lianas hacen que el movimiento sea menos predecible. Los túneles conectan varias casillas: si una entidad pisa un túnel, puede aparecer en otro túnel del mapa. Las lianas, por su parte, intentan mover una casilla más cerca de la salida, siempre que la casilla a la que se va a saltar sea válida. Como no se puede cambiar el tipo de casilla directamente, se usan conjuntos de coordenadas y funciones que revisan “después de moverse” si el jugador o un enemigo terminó en una casilla marcada como túnel o liana. Esta forma de separar la lógica especial de la grilla principal se relaciona con la idea de mantener cada módulo con un solo tipo de responsabilidad [3].

Persistencia de puntajes y uso de JSON:

El juego guarda los mejores puntajes en un archivo, de forma que, al cerrar y volver a abrir el programa, el Top 5 siga estando ahí. Para esto se utiliza JSON. Según el estándar ECMA-404 y el RFC 8259, JSON es un formato de texto sencillo basado en llaves, listas y pares clave–valor, fácil de leer para personas y programas [5], [6].

En el código, el módulo puntuación usa la librería estándar json de Python para abrir un archivo, cargar la lista de puntajes, actualizarla y guardarla de nuevo. Cuando se registra un nuevo puntaje, se lee el archivo (si no existe, se crea), se añade el nuevo registro, se ordena la lista y se dejan solo los cinco mejores. Estas operaciones siguen el estilo de los ejemplos de lectura y escritura de archivos que se muestran en la documentación de Python [2], pero adaptados a las necesidades del juego. Usar JSON evita inventar un formato propio y es más ligero que usar una base de datos para algo tan pequeño.

Herramientas de ingeniería y control de versiones:

Durante todo el desarrollo se usó Git como sistema de control de versiones y GitHub como repositorio remoto. Git permite crear commits, ver el historial de cambios, trabajar con ramas y fusionarlas más adelante, tal y como se explica en su documentación oficial y en los apartados de “Basic Branching and Merging” [7]. En este proyecto, esto permitió:

- Tener una etapa inicial con el juego en consola.
- Luego crear otra rama para la versión con Pygame.
- Ir agregando después funciones extra como el HUD, la música, los túneles y las lianas.

Si algo salía mal, se podía volver a un commit anterior sin perder el trabajo ya hecho. Además, GitHub sirve como respaldo en la nube y como evidencia clara de cómo fue avanzando el proyecto.

Problemas principales y decisiones técnicas

Uno de los primeros retos fue pasar del juego en consola a la versión con interfaz gráfica. Gracias a que la lógica estaba organizada en clases (Mapa, Jugador, Enemigo), no fue necesario reescribir todo: se mantuvo la misma “lógica del mundo” y solo se cambió cómo se muestra y cómo se lee la entrada, usando el bucle de juego y los eventos de Pygame en lugar de print e input [1]–[4].

Otro problema fue que el tipo de casilla (codigo) no se podía cambiar porque es una propiedad sin setter. Al principio se intentó modificarla directamente para convertir caminos en túneles o lianas, lo que generó errores. La solución fue mover esa responsabilidad al módulo terrenos y almacenar las posiciones de túneles y lianas en conjuntos, aplicando los efectos después del movimiento. Esta decisión respeta la interfaz original de las clases y aprovecha las recomendaciones de usar estructuras de datos adecuadas para cada tarea [2], [3].

También hubo que adaptar el sistema de energía y correr que ya funcionaba en la versión de consola para que se viera correctamente en el HUD y se actualizara con el tiempo de juego sin depender de la velocidad del computador. Para esto se usaron los relojes (Clock) de Pygame, que permiten controlar los FPS y trabajar con tiempos en segundos, tal como se sugiere en la documentación de la librería [4]. Finalmente, el sistema de puntajes con JSON se escribió de forma que maneje también casos como un archivo inexistente o dañado, usando manejo de excepciones y ejemplos de acceso a archivos de la propia documentación de Python [2], [5], [6].

Referencias (formato IEEE)

[1] Python Software Foundation, “9. Classes,” in *Python 3.14.0 Documentation*, 2025.

Available: <https://docs.python.org/3/tutorial/classes.html>

[2] Python Software Foundation, “The Python Tutorial,” in *Python 3.14.0 Documentation*, 2025. Available: <https://docs.python.org/3/tutorial/>

[3] D. Amos, “Object-Oriented Programming (OOP) in Python,” *Real Python*, 2024. Available: <https://realpython.com/python3-object-oriented-programming/>

- [4] Pygame Community, "Pygame Documentation and Tutorials," *Pygame*, 2019–2024.
Available: <https://www.pygame.org/docs/>
- [5] Ecma International, *ECMA-404: The JSON Data Interchange Syntax*, 2nd ed., 2017.
Available: <https://www.ecma-international.org/publications-and-standards/standards/ecma-404/>
- [6] T. Bray, "The JavaScript Object Notation (JSON) Data Interchange Format," RFC 8259, IETF, 2017. Available: <https://www.rfc-editor.org/rfc/rfc8259>
- [7] Git Community, "Git Reference Manual" and "Basic Branching and Merging," *Git SCM*, 2025. Available: <https://git-scm.com/doc>